

*runtime* means the error happened while the program was running as opposed to when the program was being compiled into bytecode.)

The Python standard library's `timeit` module can measure the runtime speed of a small snippet of code by running it thousands or millions of times, letting you determine an average runtime. The `timeit` module also temporarily disables the automatic garbage collector to ensure more consistent runtimes. If you want to test multiple lines, you can pass a multiline code string or separate the code lines using semicolons:

---

```
>>> import timeit
>>> timeit.timeit('a, b = 42, 101; a = a ^ b; b = a ^ b; a = a ^ b')
0.13077666299999998
>>> timeit.timeit("""a, b = 42, 101
... a = a ^ b
... b = a ^ b
... a = a ^ b""")
0.13515726800000039
```

---

On my computer, the XOR algorithm takes roughly one-tenth of a second to run this code. Is this fast? Let's compare it to some integer swapping code that uses a third temporary variable:

---

```
>>> import timeit
>>> timeit.timeit('a, b = 42, 101; temp = a; a = b; b = temp')
0.027540389999998638
```

---

That's a surprise! Not only is using a third temporary variable more readable, but it's also twice as fast! The *clever* XOR trick might save a few bytes of memory but at the expense of speed and code readability. Sacrificing code readability to reduce a few bytes of memory usage or nanoseconds of runtime isn't worthwhile.

Better still, you can swap two variables using the *multiple assignment trick*, also called *iterable unpacking*, which also runs in a small amount of time:

---

```
>>> timeit.timeit('a, b = 42, 101; a, b = b, a')
0.0244892360000007963
```

---